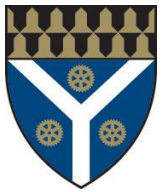




Extending TRIE for Finding Invariants in Rust Crates

Sidney Richardson, Computer Science, Ramla Ijaz and Lin Zhong, Yale University



Introduction

Much recent work applies LLMs to Rust code, but true program equivalence requires preserving deeper invariants such as temporal order and state consistency. These invariants are often implicit, encoded through Rust's affine type system, ownership rules, and the tpestate pattern, making them hard for automated tools to detect. This project extends the TRIE framework to automatically surface these hidden invariants by combining code navigation, static analysis, and language-model reasoning. By identifying tpestate transitions within a crate, the system enables AI models to understand Rust programs beyond surface-level syntax.

Project Summary

- Built an autonomous system that automatically extracts tpestate-style invariants from Rust code.
- Designed a three-stage pipeline (selection, detection, validation) that grounds every inference in concrete code lines.
- Implemented four agent iterations, each improving recall, grounding accuracy, and output stability.
- Developed a scoring framework to evaluate invariants, transitions, hallucinations, and correctness.
- Demonstrated effectiveness by recovering key implicit invariants from the OnceCell crate.
- Integrated embedding-based Rust code navigation using Octocode for structured retrieval.
- Identified clear pathways for future improvements in scalability, graph-based navigation, deduplication, and transparency.

```
enum Closed {}
enum Open {}

struct File<S> { name: String }

impl File<Closed> {
    fn open(self) -> File<Open> { .. }
}

impl File<Open> {
    fn read(&self) { ... }
    fn close(self) -> File<Closed> { .. }
}
```

```
{
  "entity": "File",
  "state": "Closed",
  "invariants": ["cannot read", "can be opened"],
  "transitions": ["Closed -> Open via open"],
  "evidence_lines": [3, 6]
},
{
  "entity": "File",
  "state": "Open",
  "invariants": ["can read", "can be closed"],
  "transitions": ["Open -> Closed via close"],
  "evidence_lines": [8, 10]
}
```

Example Tpestate Encoding

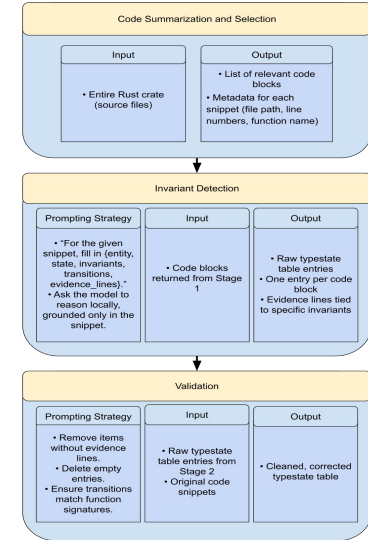
Rust's Type System

- Enforces safety at compile time through an affine type system: each value can be used at most once.
- Ownership, borrowing, and lifetimes prevent double frees, dangling pointers, and data races.
- These rules implicitly encode higher-level invariants about how data may be created, accessed, and released.

The Tpestate Pattern

- Encodes an object's runtime state in its compile-time type.
- State-specific operations: methods only available in certain states.
- Type-level state encoding: invalid operations fail at compile time.
- State transitions: consuming methods return a new value in a new state.
- Guarantees temporal order and state consistency without runtime checks.

Design and Implementation



Developmental Results

Takeaway: Higher recall of invariants raises hallucination risk; grounding tools (qualified names, reasoning steps, filtering) are critical for accurate, interpretable tpestate extraction.

- v1: Retrieved relevant code but found few invariants; many entries were incomplete or duplicated.
- v2: Stronger semantic search improved recall but introduced some ungrounded entries.
- v3: Added qualified names and reasoning steps, producing fuller tables and clearer transitions.
- v4: Introduced de-hallucination, refining entity names and filtering unsupported transitions for more stable outputs.

Future Work

- Test on more Rust crates for better scalability.
- Improve navigation with iterative or multi-pass retrieval.
- Reintroduce graph-based search once stable.
- Add deduplication to reduce noise.
- Model should include brief reasoning notes for clarity.

Evaluation

- OnceCell is a clean test case with two implicit states—Empty and Initialized—and strict invariants about when reads, writes, and initialization are allowed.
- Running the pipeline on the full crate surfaced the key patterns: atomic state transitions, initialization constraints, and value-present vs. value-absent behavior.
- The system successfully identified these implicit invariants, tied them to specific code lines, and produced stable tpestate summaries that match the crate's actual behavior.

Ver.	Invariant	Hallucination	Wrong	Transition Completeness	Duplic.	NameErr	Total
1	4	2	0	2	2	1	4
2	16	1	2	2	1	1	35.5
3	44	11	2	2	3	6	73.5
4	32	4	0	5	2	1	72

Results for each agent version